Day 2 build runbook

Eight steps, 09:15 to 14:00. The code here is written against the files you already have — `config.py`, `tools.py`, `agent.py`, `llm.py` — and the two new files keep the merge surface small. Type it tomorrow, not tonight.

Step 1 (09:15) — measure, and stop the rate limit killing you

Start the stack and the sweep before anything else. Both take time and both run unattended.

```
cd ../opentelemetry-demo && dc up -d --remove-orphans && cd -
sleep 120
.venv/bin/python run_all.py 2>&1 | tee runs/day2-before.log
```

That gives you the real before-number. Your last measured score is from 16:35 yesterday, against a `tools.py` you have since changed.

Then the fallback key, in `llm.py`. This is the twenty minutes that protects thirty points:

```
# llm.py
_fallback_in_use = False


def _switch_to_fallback():
    # called once, when the primary key is rate limited beyond its retries.
    # the team form must declare every model used, so this prints loudly.
    global _client, _fallback_in_use
    if _fallback_in_use or not config.LLM_API_KEY_FALLBACK:
        return False
    print("  PRIMARY KEY EXHAUSTED -> switching to fallback model %s" %
config.LLM_MODEL_FALLBACK,
          file=sys.stderr, flush=True)
    _client = OpenAI(api_key=config.LLM_API_KEY_FALLBACK,
                     base_url=config.LLM_BASE_URL_FALLBACK,
                     timeout=90, max_retries=3)
    config.LLM_MODEL = config.LLM_MODEL_FALLBACK
    _fallback_in_use = True
    return True
```

Then inside `chat`, in the `except TRANSIENT_ERRORS` branch, before sleeping:

```
        except TRANSIENT_ERRORS as error:
            # a rate limit that survived two waits is a daily cap, not a
burst.
            # waiting longer will not fix it; a different key will.
            if isinstance(error, openai.RateLimitError) and attempt >= 2 and
_switch_to_fallback():
                continue
            if wait_seconds is None:
                raise
```

Add the four variables to `config.py` reading from the environment, and to `.env.local`. Record in the transcript which model answered each call — one extra key in the log line — so the team form is accurate and so you can tell a judge exactly where each answer came from.

Step 2 (09:30) — the baseline fix

Three edits to `tools.py`, thirty minutes, one person. This is the half hour that is worth more than the rest of the day.

2a. Fail closed on a missing baseline. Replace `_is_slower_than_baseline` entirely:

```
def _is_slower_than_baseline(row):
    # slow means slow FOR THIS SERVICE. Accounting sits at 15s normally
because it is a
    # Kafka batch consumer; calling that an incident is how we lost three
practice runs.
    p95, baseline = row["p95_latency_ms"], row["p95_baseline_ms"]
    if not p95 or p95 < 500:
        return False
    if baseline is None:
        return False          # no baseline = we do not know this service's
normal, so this is NOT evidence
    return p95 >= 2 * baseline
```

And where `status_hint` is set, give the no-baseline case its own honest label instead of letting it fall through to `ok`:

```
        elif row["p95_baseline_ms"] is None and (row["p95_latency_ms"] or 0)
>= 500:
            row["status_hint"] = "unknown (no baseline for this service:
cannot tell normal from incident)"
```

2b. Rank by deviation and sort on it. Add the scorer, add the field to every row, and sort descending:

```
def _deviation_score(row):
    # how many times worse than its OWN normal is this service right now?
    # a missing baseline contributes nothing: unknown is not the same as bad.
    error_delta = 0.0
    if row["error_ratio_baseline"] is not None:
        error_delta = max(0.0, row["error_ratio"] -
row["error_ratio_baseline"])
    latency_excess = 0.0
    if row["p95_latency_ms"] and row["p95_baseline_ms"]:
        latency_excess = max(0.0, row["p95_latency_ms"] /
max(row["p95_baseline_ms"], 1.0) - 1.0)
    # errors weigh far more than latency: a service that started failing
beats one that got slower
    return round(max(error_delta * 20.0, latency_excess), 3)
```

Then in `get_service_health`, after the rows are built:

```
    for row in rows.values():
        row["deviation_score"] = _deviation_score(row)

    ordered = sorted(rows.values(),
                     key=lambda r:
(r["status_hint"].startswith("NOT_REPORTING"), r["deviation_score"]),
                     reverse=True)
```

And change the tool's `note` so the model reads the ranking the way you mean it:

```
    "note": ("Ranked by deviation_score = how far each service has moved from
its OWN baseline. "
             "A large absolute latency with deviation_score near 0 is normal
for that service and is "
             "NOT evidence of this incident. The top row is the strongest
candidate.")
```

2c. Stop `get_container_health` burying the answer. The `busiest_cpu` footnote held the correct service in two of the three failures. Promote anomalies into the problems list, and never report an empty problems list when something is running hot:

```
    for row in container_rows:
        cpu_now, cpu_base = row["cpu_cores_used"] or 0, row["cpu_baseline"]
or 0
        if cpu_base and cpu_now >= 3 * cpu_base and cpu_now >= 0.5:
            row["problems"].append("CPU_SPIKE (%.2f cores vs %.2f baseline,
%.1fx)"
                                   % (cpu_now, cpu_base, cpu_now / cpu_base))
        memory_now = row.get("memory_pct_of_limit") or 0
        memory_then = row.get("memory_pct_at_window_start") or 0
        if memory_now >= 70:
            row["problems"].append("MEMORY_HIGH (%.0f%% of limit)" %
memory_now)
        elif memory_then and memory_now - memory_then >= 10:
            # a cache leak is a rising line long before it is a high number
            row["problems"].append("MEMORY_CLIMBING (%.0f%% -> %.0f%% across
the window)"
                                   % (memory_then, memory_now))
```

Add `memory_pct_at_window_start` with one more Prometheus collect, the same query offset to the start of the window. Then the memory leak is visible before the OOM kill, rather than only as the `JUST_RESTARTED` aftermath you already catch.

Test it in ninety seconds, without the LLM. This is why `tools.py` has a CLI:

```
.venv/bin/python practice/faultctl.py inject ad-high-cpu
sleep 120
.venv/bin/python tools.py get_service_health      # accounting must NOT be
top
.venv/bin/python tools.py get_container_health    # ad must be in
containers_with_problems
.venv/bin/python practice/faultctl.py reset
```

If `ad` appears in the problems list and `accounting` is not ranked first, the fix has landed. Only then move on.

Step 3 (10:00) — `actions.py`

A new file, so nobody is editing `agent.py` at the same time as you. Four rules it must enforce, all in code: only allowlisted actions, never without approval, every call logged, and every action knows how to undo itself.

```
# actions.py - the ONLY way this agent changes the demo.
# Nothing here runs without an explicit approved=True, and every call is
appended to the log.
import json
```

```
import subprocess
import time

import requests

import config

ACTION_LOG = config.REPO_ROOT / "action_log.jsonl"
KILL_SWITCH_FILE = config.REPO_ROOT / "KILL_SWITCH"

# the allowlist IS the safety boundary. An action not in this dict cannot be
executed,
# whatever the model asks for.
ALLOWED_ACTIONS = {"restart_service", "toggle_feature_flag", "throttle_load",
"scale_service"}


def _append_to_log(record):
    record["logged_at"] = time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime())
    with ACTION_LOG.open("a") as log_file:
        log_file.write(json.dumps(record) + "\n")


def describe(action, target, parameters=None):
    """Dry run: exactly what WOULD change. Nothing is executed. Safe to call
always."""
    parameters = parameters or {}
    if action not in ALLOWED_ACTIONS:
        return {"ok": False, "reason": "%s is not an allowlisted action" %
action}
    if action == "restart_service":
        return {"ok": True, "command": "docker restart %s" % target,
                "effect": "Container %s stops and starts. In-flight requests
to it fail for a few seconds."
                          % target,
                "reversible": True, "undo": "none needed; a restart is not a
config change"}
    if action == "toggle_feature_flag":
        return {"ok": True, "command": "POST flag %s -> %s" % (target,
parameters.get("variant", "off")),
                "effect": "Sets flag %s to %s. WRITE ONLY: we never read flag
state to diagnose."
                          % (target, parameters.get("variant", "off")),
                "reversible": True, "undo": "set %s back to its previous
variant" % target}
    if action == "throttle_load":
```

```
        return {"ok": True, "command": "POST /swarm user_count=%s" %
parameters.get("user_count", 1),
                "effect": "Load generator drops to %s virtual users." %
parameters.get("user_count", 1),
                "reversible": True, "undo": "restore the previous
user_count"}
    # scale_service is Kubernetes-only per the Action API. We run Docker
Compose.
    # Saying so honestly is worth more than faking it.
    return {"ok": False,
            "reason": "scale_service requires Kubernetes; this deployment is
Docker Compose"}


def execute(action, target, parameters=None, approved=False,
incident_id=None):
    """Executes ONLY with approved=True. Returns a result dict; never raises
into the agent loop."""
    parameters = parameters or {}
    plan = describe(action, target, parameters)

    if KILL_SWITCH_FILE.exists():
        result = {"ok": False, "reason": "KILL_SWITCH present: refusing to
act"}
    elif not approved:
        # the gate that matters. The agent cannot set this itself; only the
human prompt does.
        result = {"ok": False, "reason": "not approved by a human: refusing
to act"}
    elif not plan["ok"]:
        result = plan
    else:
        result = _run(action, target, parameters)

    _append_to_log({"incident_id": incident_id, "action": action, "target":
target,
                    "parameters": parameters, "approved": approved,
                    "planned": plan, "result": result})
    return result


def _run(action, target, parameters):
    try:
        if action == "restart_service":
            completed = subprocess.run(["docker", "restart", target],
                                       capture_output=True, text=True,
```

```
timeout=60)
            return {"ok": completed.returncode == 0,
                    "stdout": completed.stdout.strip(),
                    "stderr": completed.stderr.strip()}
        if action == "toggle_feature_flag":
            return _write_flag(target, parameters.get("variant", "off"))
        if action == "throttle_load":
            response = requests.post(config.LOADGEN_BASE_URL + "/swarm",
                                     data={"user_count":
parameters.get("user_count", 1),
                                           "spawn_rate":
parameters.get("spawn_rate", 1)},
                                     timeout=15)
            return {"ok": response.status_code == 200, "body":
response.text[:200]}
    except Exception as error:
        # a failed action must not kill the investigation; it must be
reported and rolled back
        return {"ok": False, "reason": "%s: %s" % (type(error).__name__,
error)}
    return {"ok": False, "reason": "unreachable"}


# every action's inverse, so a failed recovery can be undone (the brief:
"roll back and escalate")
def inverse_of(action, target, parameters, before_state):
    if action == "toggle_feature_flag":
        return ("toggle_feature_flag", target, {"variant":
before_state.get("variant", "off")})
    if action == "throttle_load":
        return ("throttle_load", target, {"user_count":
before_state.get("user_count", 5)})
    return None          # a restart has no inverse: the container is already
back up
```

`_write_flag` is a straight lift from `practice/faultctl.py`. You already wrote it, it already works, and reusing it is honest — note in the README that the action path shares the flag-writing helper with the practice injector, and that neither reads a flag to make a decision.

`config.LOADGEN_BASE_URL` comes from `docker compose port load-generator 8089`, the same ephemeral-port discovery you already do for OpenSearch. Do **not** assume `localhost:8080/loadgen/` — that route is not in the demo's Envoy config on this version.

Test it before wiring anything:

```
.venv/bin/python -c "import actions;
print(actions.describe('restart_service','payment'))"
.venv/bin/python -c "import actions;
print(actions.execute('restart_service','payment'))"   # must REFUSE
.venv/bin/python -c "import actions;
print(actions.execute('restart_service','payment',approved=True))"
cat action_log.jsonl
```

The second line refusing is the proof that the gate works, and it is a good thing to show a mentor.

Step 4 (10:30) — `verify.py`

The brief: *"Execute the approved action, then check health metrics to confirm the system recovered. If it did not, roll back and escalate."* Verification is not "print recovered" — it is a comparison against a number you captured **before** you acted.

```
# verify.py - did the action actually fix it? Compared against the state
before we acted.
import time

import tools


def snapshot(service):
    """Capture the one number we will judge recovery by. Call BEFORE
acting."""
    health = tools.get_service_health(window_minutes=5)
    for row in health["services"]:
        if row["service"] == service:
            return {"service": service,
                    "error_ratio": row["error_ratio"],
                    "p95_latency_ms": row["p95_latency_ms"],
                    "deviation_score": row.get("deviation_score"),
                    "captured_at": health["window"]["to"]}
    return {"service": service, "error_ratio": None, "p95_latency_ms": None,
            "deviation_score": None, "captured_at": health["window"]["to"],
            "note": "service not reporting at snapshot time"}


def confirm_recovery(before, wait_seconds=90):
    """Wait for the metrics to catch up, then compare. Honest about 'cannot
tell'."""
    # Prometheus scrapes every 60s, so anything under ~90s is measuring the
old window
```

```
    time.sleep(wait_seconds)
    after = snapshot(before["service"])

    before_errors = before["error_ratio"]
    after_errors = after["error_ratio"]
    if before_errors is None or after_errors is None:
        return {"recovered": None, "before": before, "after": after,
                "verdict": "cannot tell: the service was not reporting at one
of the two checks"}

    # recovered = errors fell by at least 80%, or are now under 1%
    improved = after_errors <= max(before_errors * 0.2, 0.01)
    return {"recovered": bool(improved), "before": before, "after": after,
            "verdict": ("error ratio %.4f -> %.4f over %ds"
                        % (before_errors, after_errors, wait_seconds))}
```

Three things this gets right that a quick version would not.

It waits 90 seconds. Prometheus scrapes on a 60-second interval, which you measured yesterday. Checking at 10 seconds reads the pre-action window and reports a false failure. Say this out loud in the demo — it shows you understand your own telemetry.

It returns `None` for "cannot tell", not `False`. A service that stopped reporting is not the same as a service that did not recover, and conflating them would make your agent dishonest in exactly the way the brief penalises.

It compares against a captured number, not a threshold. "Error ratio 0.0999 to 0.0012" is evidence. "Looks fine now" is not.

Step 5 (10:50) — wire it into `agent.py`

One new function, called after `diagnose` returns and before the report is written. Keep it separate from `run_investigation` so the diagnosis path stays exactly as it is — it works, and you are not touching it.

```
# agent.py
import actions
import verify


def act_on_diagnosis(diagnosis, auto_approve=False):
    """Propose -> approve -> act -> verify -> (roll back). Returns the loop
record for the report."""
    proposal = diagnosis["proposed_action"]
    action, target = proposal["action"], proposal["target"]
    parameters = proposal.get("parameters", {})
```

```
    record = {"proposed": proposal}

    if action == "escalate_to_human":
        record["outcome"] = "escalated: the agent was not confident enough to
propose a change"
        return record

    if KILL_SWITCH_FILE.exists():
        record["outcome"] = "halted: KILL_SWITCH present"
        return record

    # 1. DRY RUN - always, and always shown to the human before the question
is asked
    plan = actions.describe(action, target, parameters)
    record["dry_run"] = plan
    print("\n--- PROPOSED ACTION (nothing has changed yet) ---")
    print("  action : %s on %s" % (action, target))
    print("  reason : %s" % proposal["reason"])
    print("  would  : %s" % plan.get("effect", plan.get("reason")))
    print("  undo   : %s" % plan.get("undo", "n/a"))
    if not plan["ok"]:
        record["outcome"] = "not executable: " + plan["reason"]
        return record

    # 2. APPROVAL - the only place approved becomes True
    answer = "y" if auto_approve else input("\nApprove this action? [y/N]
").strip().lower()
    record["approved"] = answer == "y"
    if not record["approved"]:
        record["outcome"] = "rejected by the human: no change was made"
        return record

    # 3. SNAPSHOT, then ACT
    before = verify.snapshot(diagnosis["root_cause"]["service"])
    record["before"] = before
    record["result"] = actions.execute(action, target, parameters,
                                       approved=True,
incident_id=diagnosis["incident_id"])

    # 4. VERIFY
    record["verification"] = verify.confirm_recovery(before)

    # 5. ROLL BACK if it did not work
    if record["verification"]["recovered"] is not True:
        undo = actions.inverse_of(action, target, parameters, before)
        if undo:
```

```
            record["rollback"] = actions.execute(undo[0], undo[1], undo[2],
approved=True,

incident_id=diagnosis["incident_id"])
            record["outcome"] = "did not recover: rolled back and escalating
to a human"
        else:
            record["outcome"] = "did not recover: nothing to roll back,
escalating to a human"
    else:
        record["outcome"] = "recovered and verified"
    return record
```

Three things to be careful about.

`auto_approve` exists only so `run_all.py` can sweep unattended. Give it a loud default of `False`, never expose it on the main CLI path, and say in the README that the sweep harness uses it and the agent never can. A judge who finds an auto-approve flag you did not mention will assume the worst.

The kill switch is checked twice — here, and again inside `actions.execute`. That is deliberate belt-and-braces, and it means a kill switch created *during* the 90-second verification wait still blocks the rollback.

Add `--yes` to the CLI for the sweep, and nothing else. Then wire it in `main()`:

```
    result = diagnose(alert_text, incident_id=args.incident_id,
out_dir=args.out)
    if not args.no_act:
        result["action_loop"] = act_on_diagnosis(result,
auto_approve=args.yes)
        # rewrite diagnosis.json and report.md now that the loop has run
        write_outputs(result, out_dir)
```

The action loop record goes into the report under its own heading: proposed, dry run, approved by whom, executed, verified, rolled back. That section **is** your Checkpoint 3 evidence.

Step 6 (11:20) — the reviewer

Stretch goal 2, thirty minutes, and it is aimed straight at your three confidently-wrong answers. One extra LLM call after the diagnosis validates, before the human sees anything.

```
# agent.py
REVIEWER_PROMPT = """You are reviewing another engineer's incident diagnosis.
Your job is to
disprove it, not to agree with it.

You see only the evidence that was actually collected and the conclusion
drawn from it.
Find the strongest reason the conclusion is wrong. Consider in particular:
- a service that is merely downstream of the real fault, or merely a client
of it
- a service whose numbers look bad but are normal FOR THAT SERVICE (check
deviation_score)
- evidence that is consistent with the conclusion but does not actually
support it

Reply with exactly one of:
CONFIRMED: <one sentence on why the evidence does support this conclusion>
DISPUTED: <the flaw> | better candidate: <service name, or none>"""


def review_diagnosis(diagnosis, evidence_records):
    """A second opinion that is paid to disagree. Never raises: a failed
review is not a failed run."""
    summary = json.dumps({
        "claimed_root_cause": diagnosis["root_cause"],
        "confidence": diagnosis["confidence"],
        "evidence": evidence_records,
        "symptom_services": diagnosis.get("symptom_services", []),
    }, indent=2)[:6000]
    try:
        reply = llm.chat([{"role": "system", "content": REVIEWER_PROMPT},
                          {"role": "user", "content": summary}])
        verdict_text = (reply.content or "").strip()
    except Exception as error:
        return {"ran": False, "reason": "%s: %s" % (type(error).__name__,
error)}
    return {"ran": True,
            "disputed": verdict_text.upper().startswith("DISPUTED"),
            "verdict": verdict_text[:500]}
```

Then apply it in `build_diagnosis`, after calibration and before the return:

```
    review = review_diagnosis(diagnosis, evidence_records)
    diagnosis["review"] = review
    if review.get("disputed"):
        # a second model found a hole. That is exactly when a confident
answer is dangerous.
        notes.append("confidence lowered one band: the reviewer disputed this
diagnosis")
        diagnosis["confidence"] = min(diagnosis["confidence"], 0.6)
        diagnosis["confidence_label"] = _label_for(diagnosis["confidence"])
```

Why this is worth thirty minutes of a two-and-a-half-hour budget. Every one of your wrong answers came back at 0.85. A reviewer looking at *"root cause: accounting, evidence: p95 15000ms"* with the deviation score now in front of it has a very good chance of saying `DISPUTED: accounting always runs at 15s | better candidate: ad`. Even when it does not catch the error, dropping to 0.6 turns a confident wrong answer into a hedged one, and the brief scores those very differently.

Put the reviewer's verdict in the report whether it agreed or not. An agent that shows you the argument it had with itself is the most convincing thing in the room, and it costs one extra paragraph of `report.py`.

Budget guard: one reviewer call per investigation, never in a loop. If the sweep is burning tokens, the reviewer is the first thing to disable with a flag — not the thing to remove.

Step 7 (12:00) — two cheap wins during harden

Both of these are allowed after 12:00 because neither is a new feature: one reads files you already wrote, the other is a demo affordance.

7a. The calibration table. Ten minutes, and it is the strongest artefact you can hand a judge scoring honesty.

```
# calibration.py - does our stated confidence match how often we are right?
# Reads the sweep tables we already produce. No new data, no LLM.
import json
import pathlib

import config

BANDS = [("high", 0.75, 1.01), ("medium", 0.5, 0.75), ("low", 0.0, 0.5)]


def build_table(reports_dir=None):
    reports_dir = pathlib.Path(reports_dir or config.REPO_ROOT / "reports")
    counts = {name: {"runs": 0, "correct": 0} for name, _, _ in BANDS}
```

```
    for diagnosis_file in reports_dir.glob("sweep-*/*/diagnosis.json"):
        diagnosis = json.loads(diagnosis_file.read_text())
        verdict_file = diagnosis_file.with_name("verdict.json")
        if not verdict_file.exists():
            continue                    # run_all.py writes this alongside
each diagnosis
        was_correct = json.loads(verdict_file.read_text())["correct"]
        confidence = diagnosis.get("confidence", 0)
        for name, low, high in BANDS:
            if low <= confidence < high:
                counts[name]["runs"] += 1
                counts[name]["correct"] += 1 if was_correct else 0

    lines = ["| stated confidence | runs | correct | actual accuracy |", "|--
-|---|---|---|"]
    for name, _, _ in BANDS:
        runs, correct = counts[name]["runs"], counts[name]["correct"]
        accuracy = "%.0f%%" % (100.0 * correct / runs) if runs else "—"
        lines.append("| %s | %d | %d | %s |" % (name, runs, correct,
accuracy))
    return "\n".join(lines)


if __name__ == "__main__":
    print(build_table())
```

Have `run_all.py` write a two-line `verdict.json` next to each diagnosis as it scores, then paste the table into `evaluation-note.md` under a heading that says what it means: *"when this agent says high confidence, it is right N% of the time on the practice set."*

If the table shows high confidence at 60%, **publish it anyway**. A team that measured its own miscalibration and says so scores better on honesty than a team that never looked — and the brief says so explicitly.

7b. The hallucination demo flag. Ten minutes, and it is thirty seconds of demo nobody can match.

```
# agent.py, inside build_diagnosis, right before check_cited_evidence runs
    if os.environ.get("DEMO_HALLUCINATION") == "1" and cited_tokens:
        # demo only: prove the guard is real by poisoning our own input
        cited_tokens = cited_tokens + ["trace-9999"]
        print("  [demo] injected a fake evidence reference: trace-9999")
```

Run it with `DEMO_HALLUCINATION=1 python agent.py "..."`. The registry rejects `trace-9999`, the model is told to resubmit, and the audience watches a safety control work rather than hearing you claim it does. Document the flag in the README — an undocumented flag that alters behaviour looks like something you were hiding.

Step 8 (13:00) — the five deliverables

Straight off the brief. All five in the repository before the freeze, or the rest does not count.

- [ ] **Source code.** Team repo forked from the template, with a README covering setup, how to run, **how the approval gate and kill switch work**, and known limitations. Yours already covers most of this — add the action loop, the `--yes` and `DEMO_HALLUCINATION` flags, and `scale_service` being unsupported on Compose.
- [ ] **Architecture overview, one page.** Components, the tools the agent uses, how it decides, and the safety controls. You have `docs/architecture.png` from Day 1 — add the four gates. A photo of a whiteboard is explicitly acceptable, so do not spend twenty minutes in a drawing tool.
- [ ] **Incident reports.** For the practice incidents it handled, in the JSON schema plus readable form. You already write both per run. Make sure the final sweep's reports are committed, not just the 16:35 ones.
- [ ] **Evaluation note.** Which practice incidents it got right and wrong in your own testing, and why. This is where the before/after sweeps and the calibration table go. Be specific about the three it got wrong yesterday and what you changed — a team that diagnosed its own agent's failure mode is the story you want told.
- [ ] **Team form.** Team name, members, who built which part, tools and AI models used **including AI coding assistants**, third-party libraries. Declare Groq and the exact model string, the fallback if it fired, and any assistant anyone used. Rule 2 makes this a condition of using them.

Then the mechanical checks, 13:30–14:00:

```
# every report validates
for f in reports/sweep-*/*/diagnosis.json; do .venv/bin/python report.py "$f"
>/dev/null || echo "INVALID: $f"; done

# all five members have commits
git log --format='%an' | sort -u

# the offline tests still pass
.venv/bin/python tests/test_offline.py

# clean run from nothing
cd ../opentelemetry-demo && dc down && dc up -d --remove-orphans && cd -
sleep 150
.venv/bin/python agent.py "Error rate above 5% on frontend"

# no secrets committed
git log -p | grep -i "gsk_" && echo "KEY IN HISTORY - tell the organisers"

git add -A && git commit -m "day 2 final" && git pull --rebase && git push
git tag day2-final && git push --tags
```

That last grep matters: a key was pasted in plain text during setup yesterday. If it ever reached a commit, rotate it and say so rather than hoping nobody looks.

One last thing, and it is a scored one. Rule 7: *"Be honest about what does not work. Stating a limitation costs nothing, hiding a failure in the demo loses points."* Your Day 1 README already has a genuinely good known-limitations list. Keep it, update it, and read from it in the demo. It is the cheapest points on the board and most teams will quietly delete theirs.